# GitHub reproducible workbook

YU-LIN CHUNG, 35480351

## Table of contents

**INTRODUCTION** **1**

**WORKING GUIDELINE** **2**

1. Creating a new RStudio Project and Quarto file. . . . . 2
2. Initialising the folder as a Git repository . . . . . 3
3. Creating the new branch and add the changes. . . . . 4
4. Adding data and amend the previous commit . . . . . 5
5. Creating merge conflict . . . . . 6
6. Deal with merge conflict . . . . . 6
7. Annotated tag . . . . . 9
8. Deleting branch locally and on the remote . . . . . 10
9. Show the commit log in condensed form in the terminal. . . . . 11
10. Add a simple plot and undo the commit without losing local changes . . . . 12

## INTRODUCTION

This is a reproducible workbook that demonstrates the knowledge of Git and GitHub usage. To help you understand who to use these tools efficiently for version control and collaboration, we will follow the instrctions to complete some tasks.

# WORKING GUIDELINE

### 1. Creating a new RStudio Project and Quarto file.

**First, you need to create a new RStudio Project called "ETC5513_Assignment2" and use the following command to create a simple qmd file called example.qmd.**

- The command `cd` is used to move into the project folder.

```
cd ETC5513_Assignment2
```

- The command `touch filename.qmd` create a new empty Quarto file.

```
touch example.qmd
```

- To make it be knitted into a HTML file, you need to write the YAML format of html and render it.

```
---
title: "GitHub reproducible workbook"
author: "YU-LIN CHUNG, 35480351"
format:
    html:
        toc: true
        theme: solar
---
```

- The command `ls` will list all of the file in "ETC5513_Assignment2". It can be seen that we successfully knitted the file into HTML when there is a file call "example.html"

```
zhongyulin@TKs-MacBook ETC5513_Assignment2 % ls
ETC5513_Assignment2.Rproj       example.html                    guide_files                     guide.qmd
example_files                   example.qmd                     guide.html
```

### 2. Initialising the folder as a Git repository

**After creating and rendering example.qmd, we can use the following command to initialise the project folder as a Git repository.**

- The command `git init` create a new local Git repository in the project folder.

```
git init
```

- The command `git add` will add the change into the staging area. The dot means the current directory. Therefore, `git add .` stages all changes in the current project folder.

```
git add .
```

- The command `git commit` create the first commit in the repository. We usually add the message with `-m` to remind us for our implement.

```
git commit -m "Inital update"
```

### 3. Creating the new branch and add the changes.

**Now we need to do two things: First, create a new branch called testbranch. Second, modify the file example.qmd and add the changes to both the local and remote repositories.**

- To make sure we are in the main branch. The command `git branch` can ensure where you are if you see an asterisk symbol in front of the branch name.

```
git branch
```

- The command `git switch` will create a new branch and switch you to this branch.

```
git switch -c testbranch
```

- After modifying example.qmd, we can use `git status` to check the status of it.

```
git status
```

- Following the above steps, we need to add the modified file to the staging area with the command `git add` and commit the change to the local repository with the command `git commit` and write the message for it.

```
git add example.qmd
git commit -m "Modify example.qmd on testbranch"
```

- Finally, we need to push the new branch and its commit to the remote GitHub repository by using command `git push`. However, because it is the first time to push the new branch to GitHub, we need to set up upstream with `-u`

```
git push -u origin testbranch
```

### 4. Adding data and amend the previous commit

**Next, we need to create a folder called data, and add the data from Assignment 1 to that folder.**

- We need to make sure we are working on testbranch. The command `git branch` will show where you are with the asterisk symbol in fornt of the branch name.

```
git branch
```

- The command `mkdir` create a new folder.

```
mkdir data
```

- The command `cp` is used to copy the data in to the folder "data" .

```
cp "/Users/zhongyulin/assignment-1-creating-reproducible-reports-yulinchung/data/35480351_Yu
```

- The command `cd` is used to move into specific folder and we can use `ls` to check that the file had been copied successfully.

```
cd data
ls
```

**After that, we will learn how to amend the previous commit to include the data folder and push this amended commit to the remote.**

- Instead of creating a new commit, we need to amend the previous commit. The command `git commit --amend` modifies the most recent commit. The option `-m` allows us to write a new commit message.

```
git commit --amend -m "Modify example.qmd and add data folder"
```

- Finally, we need to push it to the remote GitHub repository by using command `git push`. However, because we amended a commit that had already been pushed, the commit history changed. Therefore, we need to use `git push -f` rather than a normal `git push` to force GitHub to update the remote branch.

```
git push -f origin testbranch
```

### 5. Creating merge conflict

- First, we need to switch back to the `main` branch.

```
git switch main
```

- To create the conflict, we should types some similar content in main branch as the same position in testbranch.

```
### Changes of the testbranch
Change of the new branch call "testbranch"
```

Figure 1: content in testbranch

```
### Changes of the main branch
Change made on the main branch to create a conflict with testbranch.
```

Figure 2: content in main

- After modifying the file, we add and commit the change, and try to push the change to GitHub.

```
git add example.qmd
git commit -m "Modify example.qmd on main"
git push
```

### 6. Deal with merge conflict

- To merge `testbranch` into `main`, we use the command `git merge`

```
git merge testbranch
```

- After completing the steps above, we will find that the system reminds us there is a conflict.

```
zhongyulin@TKs-MacBook ETC5513_Assignment2 % git merge testbranch
warning: Cannot merge binary files: .DS_Store (HEAD vs. testbranch)
Auto-merging .DS_Store
CONFLICT (content): Merge conflict in .DS_Store
Auto-merging example.qmd
CONFLICT (content): Merge conflict in example.qmd
Automatic merge failed; fix conflicts and then commit the result.
zhongyulin@TKs-MacBook ETC5513_Assignment2 %
```

- We need to fix the merge conflict.

```
## INTRODUCTION
This is an example .qmd for my assignment2.

Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<<< HEAD (Current Change)
### Changes of the main branch
Change made on the main branch to create a conflict with testbranch.
=======
### Changes of the testbranch
Change of the new branch call "testbranch"
>>>>>>> testbranch (Incoming Change)
```

```
## INTRODUCTION
This is an example .qmd for my assignment2.

### Changes from both branches
This section combines the changes made on branches of main and testbranch. The
main branch was modified to create a merge conflict and testbranch was modified
earlier to demonstrate branch-based changes.
```

- After resolving the conflict, we can use `git status` to check the current status. This command shows whether all conflicts have been fixed and whether the files are ready to commit.

```
git status
```

- After the conflict was fixed, we used `git add` to add the changes into the staging area. Then, we commit the resolved merge conflict. This commit records the final version after

merging testbranch into main. Finally, we push the merge commit to the remote GitHub repository.

```
git add example.qmd
git commit -m "Resolve merge conflict"
git push
```

- After pushing, we can refresh the GitHub repository page and check the `main` branch. The latest commit message is "Resolve merge conflict", which shows that the merge result has been pushed to GitHub.

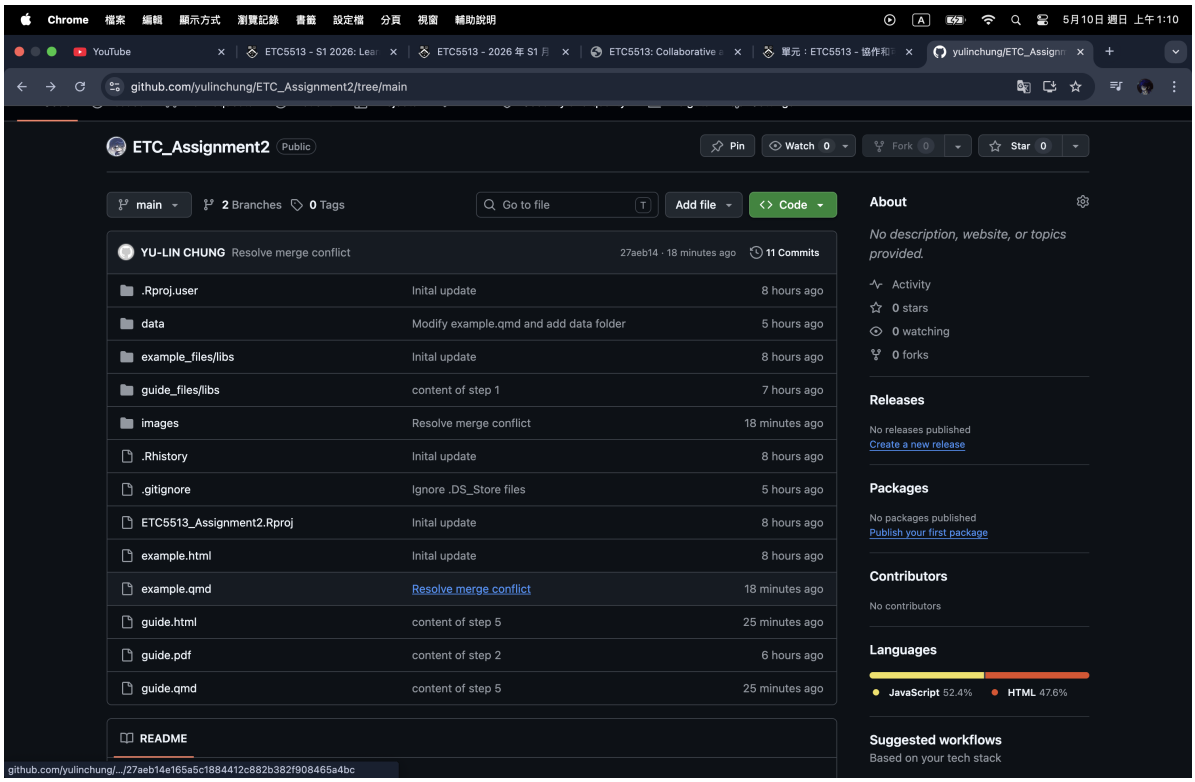


Figure 3: the GitHub status

### 7. Annotated tag

**The next step, we need to tag the commit of sloving merge conflict v1.0 on main using an annotated tag.**

- Before creating the tag, we can use `git log --oneline` to check the commit history and find the commit that we want to tag.

```
git log --oneline
```

- The command `git tag -a` means to create an annotated tag and the option `-m` allows us to write a message for the tag. Because I made another commit after resolving the merge conflict, we need to type the commit hash to make sure we add the annotated tag on the write place.

```
git tag -a v1.0 27aeb14 -m "Version 1.0"
```

- To check more details about the tag, we can use `git show`. This command shows the tag message and the commit that the tag points to.

```
git show v1.0
```

- Finally, we need to push the tag to the remote GitHub repository.

```
git push origin v1.0
```

### 8. Deleting branch locally and on the remote

**In this step, we need to delete the branch called testbranch both locally and on the remote GitHub repository.**

- First, we need to make sure we are not working on `testbranch`. We can use `git branch` to check the current branch with the asterisk symbol in front of it. The command `git switch` move us into the branch we want.

```
git branch
git switch main
```

- we can delete the local branch by using `git branch -d`.

```
git branch -d testbranch
```

- The command `git push origin --delete` deletes the branch from the remote repository.

```
git push origin --delete testbranch
```

**9. Show the commit log in condensed form in the terminal.**

**In this step, we need to show the commit history in a condensed form in the terminal. A condensed commit log is easier to read because it shows each commit on one line.**

- The command `git log --oneline` shows the commit history in a shorter format.

```
git log --oneline
```

```
zhongyulin@TKs-MacBook ETC5513_Assignment2 % git branch
* main
zhongyulin@TKs-MacBook ETC5513_Assignment2 % git log --oneline
2a8ae2c (HEAD -> main) content of step 7
aa1dc47 (origin/main) Update guide pdf
d20bc21 complete 1~6
27aeb14 (tag: v1.0) Resolve merge conflict
ad8e957 content of step 5
9388936 some conten of step 5
e224d54 Modify example.qmd on main
b34c502 content of step 4
1fbd113 Ignore .DS_Store files
7828793 Modify example.qmd and add data folder
e439c0a conten of step 3
99063a5 content of step 2
2b324b4 content of step 1
1249172 Inital update
zhongyulin@TKs-MacBook ETC5513_Assignment2 %
```

### 10. Add a simple plot and undo the commit without losing local changes

**In this step, we need to create a new section in example.qmd that includes an easy plot. In this part, we commit the change and use the command line interface to undo the commit without losing the local changes.**

- After saving example.qmd, we add the modified file to the staging area and commit the change.

```
git add example.qmd
git commit -m "Add simple plot"
```

- We can use git log –oneline to check that the commit Add simple plot has been created.

```
git log --oneline
```

- To undo the latest commit without losing the local changes, we use `git reset` to move back to a previous commit. The option `--soft` removes the most recent commit, but keeps the changes in the staging area, and `HEAD~1` means one commit before the current commit.

```
git reset --soft HEAD~1
```

- Finally, we use git status to check the result.

```
git status
```